

Компьютерная графика

Практика 3: Буферы, атрибуты вершин, кривые Безье

2026

- Используются для генерации плавных кривых

- Используются для генерации плавных кривых
- Строятся по набору точек $p_0, p_1, \dots, p_n$

- Используются для генерации плавных кривых
- Строятся по набору точек $p_0, p_1, \dots, p_n$
- Кривая Безье с параметром $t \in [0, 1]$ определяется как *аффинная комбинация*

$$b(t) = \sum_{k=0}^n b_{k,n}(t) \cdot p_k$$

- Используются для генерации плавных кривых
- Строятся по набору точек $p_0, p_1, \dots, p_n$
- Кривая Безье с параметром $t \in [0, 1]$ определяется как *аффинная комбинация*

$$b(t) = \sum_{k=0}^n b_{k,n}(t) \cdot p_k$$

- Коэффициенты – полиномы Бернштейна:

$$b_{k,n}(t) = \binom{n}{k} t^k (1-t)^{n-k}$$

Кривые Безье

- Используются для генерации плавных кривых
- Строятся по набору точек $p_0, p_1, \dots, p_n$
- Кривая Безье с параметром $t \in [0, 1]$ определяется как *аффинная комбинация*

$$b(t) = \sum_{k=0}^n b_{k,n}(t) \cdot p_k$$

- Коэффициенты – полиномы Бернштейна:

$$b_{k,n}(t) = \binom{n}{k} t^k (1-t)^{n-k}$$

- Кривая первого порядка ($n = 1$): отрезок $p_0 p_1$

$$b(t) = (1-t) \cdot p_0 + t \cdot p_1$$

Кривые Безье

- Используются для генерации плавных кривых
- Строятся по набору точек $p_0, p_1, \dots, p_n$
- Кривая Безье с параметром $t \in [0, 1]$ определяется как *аффинная комбинация*

$$b(t) = \sum_{k=0}^n b_{k,n}(t) \cdot p_k$$

- Коэффициенты – полиномы Бернштейна:

$$b_{k,n}(t) = \binom{n}{k} t^k (1-t)^{n-k}$$

- Кривая первого порядка ($n = 1$): отрезок $p_0 p_1$

$$b(t) = (1-t) \cdot p_0 + t \cdot p_1$$

- Кривая второго порядка ($n = 2$):

$$b(t) = (1-t)^2 \cdot p_0 + 2t(1-t) \cdot p_1 + t^2 \cdot p_2$$

Задание 1

Загружаем данные в буфер на GPU

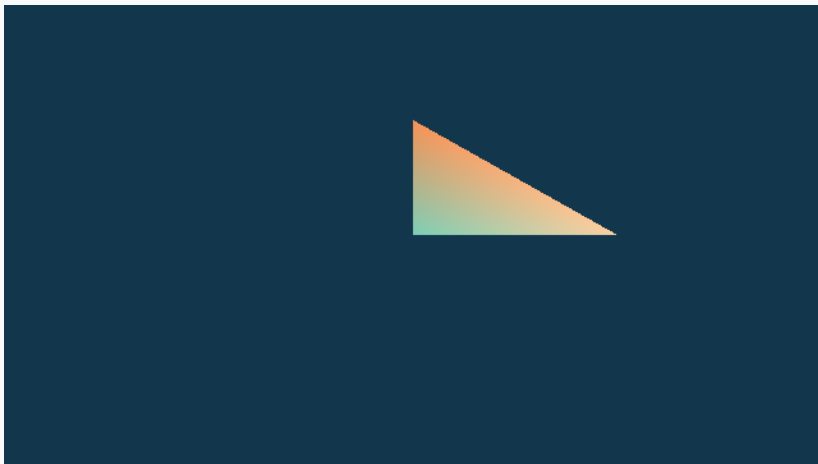
- Создайте буфер: `device.create_buffer,`
`usage = CopyDst | Vertex`, размер – 3 вершины (надо посчитать в байтах!)
- Загрузите в него данные `queue.write_buffer`
- На Rust вам понадобится `bytemuck::cast_slice(&vertices)`

Задание 2

Настраиваем атрибуты

- Настройте атрибуты вершин при создании пайплайна (offset, stride, и формат надо понять по структуре `vertex`)
- Перед `render_pass.draw` выставьте наш буфер как вершинный `render_pass.set_vertex_buffer` – должен нарисоваться треугольник

Задание 2



Переходим к оконным координатам

- Заполните массив `viewMatrix` матрицей, которая совершает проеобразование

$$X : [0, width] \rightarrow [-1, 1]$$

$$Y : [height, 0] \rightarrow [-1, 1]$$

- Измените координаты треугольника, чтобы он был заметен на экране

Задание 3

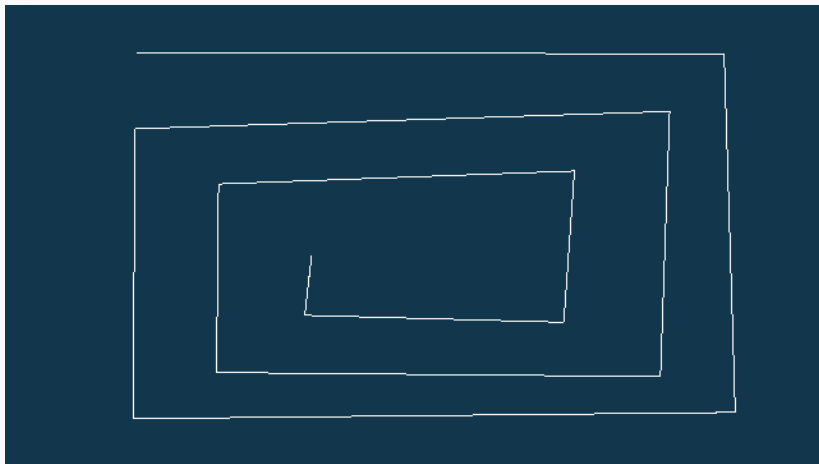


Задание 4

Динамически добавляем/удаляем точки мышкой

- Сделайте массив вершин изначально пустым
- При нажатии левой кнопки мыши добавьте новую вершину в массив с координатами мыши (цвета выбирайте как угодно)
- При нажатии правой кнопки мыши удалите последнюю вершину из массива, если он не пустой
- Если массив с вершинами изменился, обновите данные в буфере (возможно, его придётся пересоздать с большим размером)
- Меняем тип примитива у пайплайна на `LineStrip`
- Меняем количество вершин в `render_pass.draw()` на размер массива вершин
- Данные должны обновляться **только** когда линия изменилась!

Задание 4

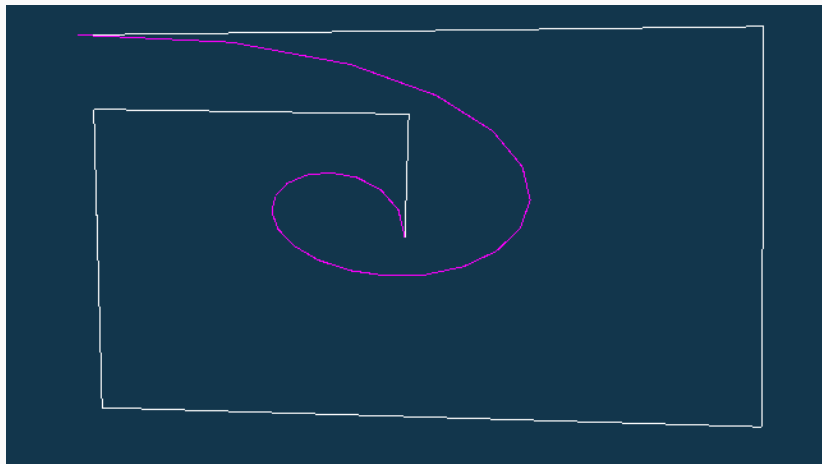


Задание 5

Генерируем и рисуем кривые Безье

- Создаёте ещё один буфер и массив вершин для точек кривой Безье
- Заведите параметр, управляющий уровнем детализации кривой: `int quality = 4;`
- При добавлении/удалении точек пересчитываем заново всю кривую Безье
- Если в исходной ломаной n отрезков, то в кривой Безье должно быть $n * quality$ отрезков
- Параметр t должен равномерно меняться от 0 до 1 вдоль всей кривой
- Цвет – любой, но отличающийся от цвета исходной ломаной (чтобы её было лучше видно)
- Данные в буферах должны обновляться **только** при их изменении
- Рисуем кривую, используя тот же пайплайн (но другой буфер)

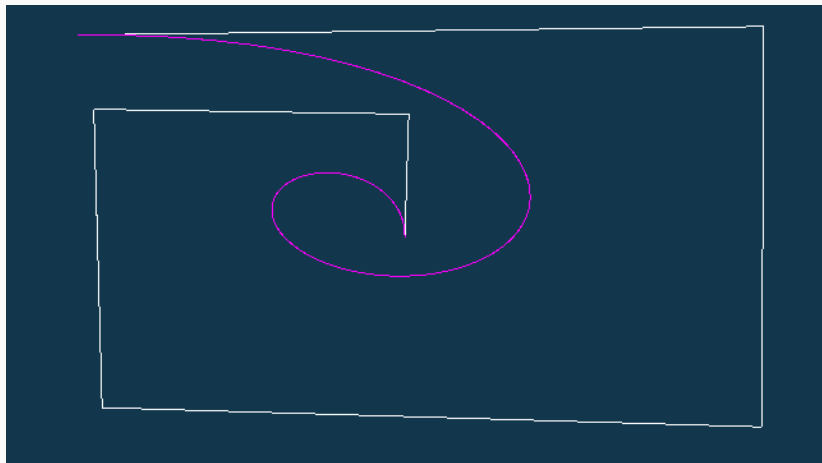
Задание 5



Динамически меняем уровень детализации

- При нажатии на стрелку влево (`SDL_LEFT`) уровень детализации `quality` должен уменьшаться (но не может быть меньше 1)
- При нажатии на стрелку вправо (`SDL_RIGHT`) уровень детализации `quality` должен увеличиваться
- Данные в буферах должны обновляться **только** при их изменении!

Задание 6



Задание 7*

Добавляем ползающий пунктир к кривой Безье

- Пунктира можно добиться, не рисуя половину пикселей кривой в зависимости от расстояния до начала
- Например, пиксели кривой на расстоянии `[0, 20]` от начала рисуются, на расстоянии `[20, 40]` от начала не рисуются, `[40, 60]` рисуются, и т.д.
- Для этого нужно добавить в вершины кривой расстояние вдоль кривой от её начала
- Расстояние до первой вершины = 0
- Расстояние до вершины `i` = расстояние до вершины `(i-1)` + расстояние от вершины `(i-1)` до вершины `i`
- Для расстояния между вершинами пригодится функция `math::length`
- Расстояние нужно интерполировать между вершинами и передать во фрагментный шейдер

Задание 7*

- Чтобы понять, нужно ли рисовать пиксель, пригодится функция WGSL `modf`
- Чтобы отменить рисование пикселя, нужно во фрагментном шейдере вызвать команду `discard`;
- Само расстояние нужно добавить как поле вершины и как атрибут в пайплайне и в вершинном шейдере
- Чтобы пунктир двигался, можно прибавить к расстоянию что-то, зависящее от времени (придётся передать время в шейдер как часть `immediates`)
- Чтобы на исходную ломаную не применялся пунктир, придётся передать ещё какой-нибудь флаг в `immediates`

Задание 7*

